

Архитектурная эволюция видеоигровых бенчмарков: движковые инновации, системная оптимизация и вычислительные компромиссы

1. Графические архитектуры: растеризация, отложенный свет, кластеризация и микрогеометрия

Эволюция графических технологий представляет собой непрерывный переход от эвристических методов пространственного отсечения к динамическому моделированию физики оптических явлений непосредственно на графическом процессоре. Каждый технологический скачок диктовался необходимостью преодоления аппаратных барьеров: дефицита пропускной способности оперативной памяти, пределов скорости заполнения сцены пикселями (fill-rate) и вычислительной сложности освещения.

На заре трехмерной графики ключевой проблемой было упорядочивание геометрии по глубине в условиях жесткого дефицита ресурсов процессоров Intel 80386/80486. В *Doom* (1993, движок id Tech 1) Джон Кармак применил математический аппарат двоичного разбиения пространства (Binary Space Partitioning, BSP). При компиляции уровня утилита заранее выстраивала 2D BSP-дерево секторов, что позволяло во время рендеринга обходить сцену от ближайших плоскостей к дальним (front-to-back) без попиксельной сортировки в кадре. Стены выводились вертикальными экранными колонками с проверкой перекрытия через 1D-буфер диапазонов, а пол и потолок заливались через алгоритм flood-fill в единые горизонтальные плоскости видимости (visplanes) с использованием целочисленной арифметики с фиксированной точкой 16.16. Архитектурной платой за скорость стала невозможность создания честной трехмерной архитектуры: сектора исключали комнаты друг над другом, вертикальный наклон камеры отсутствовал, а лимит в 128 видимых visplanes приводил к вылету движка при усложнении геометрии.

В консольном сегменте первой вехой честного трехмерного рендеринга стала *Super Mario 64* (1996, Nintendo 64). Проект разрабатывался под архитектуру чипсета Silicon Graphics Reality Coprocessor (RCP), разделенного на сигнальный процессор Reality Signal Processor (RSP) и растровый процессор Reality Display Processor (RDP). Движок опирался на микрокод Fast3D, выполнявший геометрические трансформации, расчет вершинного освещения и отсечение по усеченной пирамиде видимости (clipping). Фундаментальным аппаратным ограничением чипа RDP являлся экстремально малый размер текстурного кэша (TMEM) — ровно 4 КБ. Чтобы наложить текстуру с билинейной фильтрацией, движок был вынужден использовать микротекстуры размером 32×32 пикселя при 16-битном цвете либо 64×32 при 8-битном квантовании палитры. Для обхода этого лимита Nintendo применила процедурные шейдерные комбинации: базовые низкополигональные меши окрашивались через интерполяцию цветов вершин (Gouraud Shading) в сочетании с

наложением мелких тайловых текстур шума, что маскировало размытие текстур при экстремальном дефиците TMEM.

По мере усложнения геометрии классический прямой рендеринг (Forward Rendering) столкнулся с квадратичной сложностью вычисления динамического света: $O(N \times M)$, где N — число полигональных фрагментов, а M — число источников света. В *Metal Gear Solid V: The Phantom Pain* (Fox Engine, 2015) студия Kojima Productions применила отложенный рендеринг (Deferred Shading). Свойства поверхностей за один проход упаковывались в промежуточный G-буфер, после чего свет вычислялся как двумерные экранные объемы со сложностью:

$O(N + M)$

Для разгрузки шины памяти разработчики упаковали диффузное глобальное освещение в локальные зонды на базе сферических гармоник (Spherical Harmonics) 2-го порядка (9 коэффициентов на цветовой канал). Диффузная составляющая рассчитывалась в буфере половинного разрешения с последующим билатеральным апскейлингом с учетом карты глубин, что обеспечило стабильные 60 кадров/с на базовой PlayStation 4 ценой отказа от аппаратного сглаживания MSAA и усложнения форвардных проходов для полупрозрачных эффектов.

Противоположный кризис настиг *Assassin's Creed Unity* (2014, AnvilNext). Стремление воссоздать революционный Париж в масштабе 1:1 с детализированными фасадами привело к геометрическому взрыву: число вызовов отрисовки (draw calls) на кадр достигало 50 000. Конвейер DirectX 11, завязанный на последовательную передачу команд из одного потока CPU, захлебывался в драйверных очередях, что вызывало катастрофические просадки частоты кадров даже на флагманских ПК. Освещение сцены опиралось на массивную сетку запеченных волуметрических зондов (Light Probes), запекание которых на кластерах серверов занимало недели.

В 2020 году id Software в *DOOM Eternal* (idTech 7) отказалась от G-буфера в пользу кластеризованного прямого рендеринга (Clustered Forward Rendering). Причиной стал неприемлемый расход видеопамати под буферы геометрии в разрешении 4K (свыше 250 МБ) и потребность в сотнях полупрозрачных эффектов частиц. Архитектура движка под API Vulkan использует глубокий предварительный проход глубины (Depth Pre-pass) для раннего отсека невидимых фрагментов. Пирамида видимости нарезается на 3D-кластеры, внутри которых compute-шейдер растеризует границы источников света в тайлы с упаковкой индексов в 64-битные битовые маски. Форвардный шейдинг считывает массив видимых источников света через битовые операции без обращений к тяжелым буферам памяти. Применение Bindless-текстур и Uber-шейдеров сократило число состояний конвейера (PSO) до ~500 на всю игру, обеспечив сотни кадров в секунду ценой сложнейшей ручной синхронизации барьеров памяти GPU.

В сегменте мобильного и кроссплатформенного рендеринга эталоном стилизации стал *Genshin Impact* (2020, модифицированный Unity). Разработчики реализовали пайплайн нефотореалистичного рендеринга (NPR, Non-Photorealistic Rendering) с кастомным cel-shading. Для устранения артефактов ступенчатых теней на лицах персонажей нормали геометрии головы вручную запекались в сферические векторы, а расчет светотени производился через чтение специальной карты полутонов (LightMap/Face Map), независимой от угла падения честных полигональных лучей. Для адаптации под тайловые видеочипы смартфонов (Qualcomm Adreno, ARM Mali с архитектурой TBDR — Tile-Based Deferred Rendering) рендер-таргеты были строго минимизированы по весу, чтобы промежуточные буферы полностью помещались в быструю локальную память кристаллов On-Chip Tile Memory, исключая перегрев и троттлинг шины LPDDR.

Инди-сегмент продемонстрировал обратное движение — намеренную деконструкцию современного конвейера. В *Ultrakill* (2020, Unity) разработчик реализовал кастомный программный пайплайн, имитирующий аппаратные ограничения консоли PlayStation 1. Для воспроизведения эффекта дрожания геометрии координаты вершин в вершинном шейдере искусственно квантуются и округляются до фиксированной сетки экранных пикселей (Vertex Snapping). Текстурные координаты интерполируются линейно в экранном пространстве без деления на глубину W, что порождает характерные аффинные текстурные искажения (Texture Warping). Из рендеринга были вырезаны буферы темпорального сглаживания и тяжелые проходы пост-обработки, что свело время кадровой задержки ввода (Input Latency) к теоретическому минимуму дисплея. Переход к физически корректному переносу фотонов начался с *Metro Exodus: Enhanced Edition* (4A Engine, 2021). Разработчики полностью вырезали растровые эвристики и запеченные карты света, сделав аппаратные RT-ядра обязательными. Базой освещения стала система диффузных зондов (DDGI), скомбинированная с темпоральным накоплением. Зонды зондируют сцену лучами в мировом пространстве, моделируя бесконечное число отскоков света. Платой за физическую честность стал эффект инерции: при резком включении источника фотоны заполняют пространство с задержкой в 15–20 кадров, сопровождаясь видимым темпоральным шумом. Вершиной трассировки стала интеграция трассировки путей (Path Tracing) на базе алгоритмов ReSTIR (Reservoir Spatio-Temporal Importance Resampling) в *Cyberpunk 2077* (режим RT Overdrive) и *Black Myth: Wukong* (Unreal Engine 5). Алгоритм выпускает всего 1–2 луча на пиксель, после чего объединяет их с соседними пикселями в пространстве и предыдущими кадрами во времени через структуры резервуаров с весами правдоподобия. Для устранения расхождения лучей внутри вычислительных блоков GPU задействуется механизм Shader Execution Reordering (SER), динамически сортирующий расходящиеся потоки инструкций в когерентные варпы. Финальная картинка восстанавливается с помощью нейросетевой реконструкции лучей (Ray Reconstruction / DLSS RR), заменившей отдельные фильтры размытия единой автоэнкодерной моделью. В *Alan Wake 2* (Northlight Engine) к этому добавился отказ от устаревшего конвейера вершинных/индексных буферов в пользу сеточных шейдеров (Mesh Shaders), выполняющих отсечение геометрии силами вычислительных блоков видеокарты, что сделало запуск игры невозможным на графических процессорах без поддержки DirectX 12 Ultimate.

Проект	Движок / API	Графическая концепция	Архитектурные преимущества	Критические системные ограничения
Doom (1993)	id Tech 1 / DOS	BSP front-to-back, 1D-буфер стен, заливка visplanes	Отсутствие сортировки и z-буфера, работа на 4 МБ RAM	Отсутствие многоэтажности, лимит 128 visplanes, плоская геометрия
Super Mario 64 (1996)	Проприетарный / N64	Микрокод Fast3D, RSP/RDP конвейер, Gouraud Shading	Аппаратный 3D-клиппинг, интерполяция цветов вершин	Лимит 4 КБ ТМЕМ, необходимость микротекстур 32×32

Проект	Движок / API	Графическая концепция	Архитектурные преимущества	Критические системные ограничения
MGS V (2015)	Fox Engine / DX11	Deferred Shading + SH-зонды 2-го порядка	Сложность $O(N+M)$, сверхнизкое потребление VRAM под диффуз	Сложность полупрозрачных эффектов, отсутствие аппаратного MSAA
AC Unity (2014)	AnvilNext / DX11	Микрополигональный масштаб 1:1, запеченные волюметрические Light Probes	Беспрецедентная детализация архитектуры и плотность толпы	До 50 000 draw calls на кадр, перегруз однопоточного DX11 API
DOOM Eternal (2020)	idTech 7 / Vulkan	Clustered Forward + Depth Pre-pass + Bindless Uber-shaders	Отсутствие расхода VRAM под G-буфер, нативная прозрачность	Высокая сложность низкоуровневой барьерной синхронизации памяти GPU
Genshin Impact (2020)	Модифицированный Unity / GLES, Metal, DX11	Мобильный NPR Cel-Shading + сглаженные карты нормалей лица	Минимальный термopakет, оптимизация под TBDR-память кристаллов	Упрощенное самозатенение, зависимость от запеченных масок
Ultrakill (2020)	Unity / DX11	PS1-стилизация: Vertex Snapping, аффинные UV, отказ от буферов	Минимальная задержка кадра, работа на любом слабом GPU	Отсутствие темпоральной стабильности, намеренное дрожание картинки
Metro Exodus EE (2021)	4A Engine / DX12, Vulkan	DDGI Infinite Bounces + Temporal Radiance Cache	Полный отказ от запекания, физически честный диффузный отскок	Полная неработоспособность без RT-ядер, инерционный лаг света
Cyberpunk 2077 (2023)	REDengine 4 / DX12	Full Path Tracing + ReSTIR GI/DI + SER + Ray Reconstruction	Честный перенос фотонов от миллионов неоновых источников	Неиграбелен без апскейлинга и генерации кадров, шум в динамике
Alan Wake 2 (2023)	Northlight / DX12	Path Tracing + Mesh Shaders Geometry Pipeline	Микрополигональный субпиксельный каллинг, точный свет	Аппаратный барьер: отказ от видеокарт без DX12 Ultimate

2. Стриминг открытых миров, иерархическая видимость и I/O архитектура

Создание бесшовных виртуальных пространств исторически упиралось не в вычислительную мощность GPU, а в пропускную способность шины ввода-вывода и время случайного позиционирования считывателя накопителя данных.

Фундамент бесшовных миров заложила *Grand Theft Auto III* (2001, PlayStation 2, RenderWare). Разработчики из DMA Design были ограничены 32 МБ оперативной памяти консоли и скоростью оптического DVD-привода в 2–4 МБ/с при задержке позиционирования лазерной головки до 150–200 мс. Загрузить город размером 130 МБ целиком было невозможно.

Команда развернула вокруг камеры скользящее окно стриминга, разбитое на сектора. Чтобы предотвратить фатальную фрагментацию оперативной памяти, ведущий программист Оббе Вермей создал специализированный аллокатор: ресурсы упаковывались в монолитные блоки с шагом строго по 2 КБ, а физическая запись секторов на дорожки DVD-диска строго зеркалировала топологию виртуальных островов Либерти-Сити. Медлительность накопителя продиктовала правила левел-дизайна:

- Максимальная скорость автомобилей в коде была программно ограничена, а в физическую модель добавили скрытое сопротивление воздуха (+5% на высоких скоростях), чтобы транспорт не мог обогнать чтение данных с диска.
- Планировка улиц заставляла игрока совершать повороты под прямым углом перед зданиями-заглушками (как небоскреб в районе Сент-Маркс), предоставляя DVD-приводу дополнительные полторы секунды на позиционирование лазера.
- Полеты на самолетах были заблокированы (крылья Dodo обрезаны), так как вертикальный обзор раскрывал не загруженные сектора, а пул активного трафика ограничивался 8 моделями автомобилей одновременно.

В *Grand Theft Auto: San Andreas* (2004) разработчики выжали ядро RenderWare до предела, выделив в файле stream.ini фиксированный пул объемом 13.5 МБ под потоковую загрузку. Для объединения трех мегаполисов без видимых экранов загрузки в структуру карты были внедрены обширные сельские зоны, леса и пустыни. Они выступали географическим буфером памяти: пока персонаж пересекал сельскую местность, предыдущий город выгружался из ОЗУ, а следующий поблочно считывался с диска.

Аналогичный принцип непрерывной виртуализации носителя использовался в платформере *Crash Bandicoot* (1996, PlayStation 1). Студия Naughty Dog написала кастомную файловую систему NPT, разбиравшую уровни на чанки по 64 КБ, которые непрерывно считывались с привода CD-ROM на скорости 300 КБ/с прямо в 2 МБ системной памяти в обход стандартной операционной системы Sony, а уровни проектировались исключительно вдоль оси обзора камеры.

В *Grand Theft Auto V* (2013 на PS3/Xbox 360, 2015 на ПК) движок RAGE совершил переход к масштабируемому 64-битному стримингу. На консолях седьмого поколения с их лимитом в 512 МБ объединенной памяти Rockstar была вынуждена требовать обязательную установку части данных на жесткий диск при одновременном чтении остального массива с оптического диска, распараллеливая считывание между двумя физическими приводами. Архитектура стриминга RAGE перешла на многоуровневые радиальные кольца внимания: физические коллизии загружались в радиусе 150 метров, высокодетализированные меши — до 300 метров, а дальние горизонты обрабатывались через иерархическую систему

HLOD-импостеров (Hierarchical Level of Detail) и плоских текстурных карт нормалей, заменяющих целые горные гряды единым полигональным полотном.

В проекте *Red Dead Redemption* (2010, RAGE) инженерам пришлось адаптировать стриминг под специфическую архитектуру консоли PlayStation 3. Из-за разделения 512 МБ памяти на два изолированных пула по 256 МБ (XDR RAM для CPU и GDDR3 для GPU) декомпрессия текстур и генерация геометрии ландшафта были вынесены на микроядра SPU (Synergistic Processing Elements) процессора Cell Broadband Engine. Микроядра забирали упакованные фрагменты карты высот по каналам прямого доступа к памяти (DMA), производили тесселяцию рельефа в локальной памяти Local Store объемом 256 КБ и отправляли готовую геометрию напрямую в видеопамять, разгружая основное ядро PowerPC.

В *Red Dead Redemption 2* (2018) потоковая система RAGE эволюционировала в асинхронный конвейер Terrain Clipping: сетка рельефа масштабируется концентрическими квадратами относительно камеры с логарифмическим падением плотности вершин к горизонту. Однако возросшая детализация привела к тяжелому оверхеду на ПК: распаковка плотных потоков ассетов и декомпрессия физических данных флоры создают пиковые нагрузки на шину PCIe и контроллер памяти, требуя от процессора развитой многопоточности.

Катастрофическим примером сбоя архитектуры стриминга стал ПК-релиз *Batman: Arkham Knight* (2015, сильно модифицированный 64-битный Unreal Engine 3). В отличие от консолей PS4 и Xbox One с их быстрой унифицированной памятью, ПК-версия столкнулась с разрывом между системной памятью и VRAM. Скорость передвижения Бэтмобиля (до 150 км/ч) многократно превышала возможности стримингового менеджера UE3 по своевременной подгрузке текстур высокого разрешения. Движок пытался непрерывно переаллоцировать текстурные буферы прямо в главном графическом потоке, что приводило к провалу частоты кадров со стабильных 60 до 5–10 кадров/с и жестким графическим глитчам, вынудив издателя временно отозвать игру из продажи.

Иной компромисс демонстрирует *Mass Effect 3* (2012, Unreal Engine 3). Не имея возможности бесшовно транслировать комплексные локации цитадели в пределах 512 МБ памяти консолей, BioWare применила пространственные шлюзы декомпрессии.

Знаменитые сцены поездок на лифтах, сканирования на входе в Нормандию и шлюзовые камеры являлись замаскированными экранами блокирующей загрузки: старая подзона полностью выгружалась из ОЗУ, сборщик мусора очищал дескрипторы объектов, а с оптического диска считывался следующий уровень.

В серии *Need for Speed* (эволюция от движка EAGL в *Most Wanted* до Frostbite в *Rivals*) архитектура стриминга строилась вокруг экстремальных скоростей перемещения (свыше 350 км/ч). Чтобы исключить провалы сквозь дорожное полотно, физический меш трассы полностью отделялся от визуального рендера: геометрия коллизий дорожного полотна кэшировалась в памяти в виде непрерывной сплайновой ленты на несколько километров вперед по вектору движения, в то время как визуальные ассеты зданий и окружения могли запаздывать и подгружаться с агрессивным pop-in эффектом. Физика сцепления колес рассчитывалась по эмпирической формуле Ханса Пасейки (Pacejka Magic Formula), требовавшей стабильного шага обновления без пропусков кадров ввода.

В современных движках стриминг геометрии полностью переносится на сторону видеокарты. В движке Decima (*Horizon Zero Dawn*, *Death Stranding*) студия Guerrilla Games применила полностью управляемую GPU видимость (GPU-driven visibility). Пространство сцены организовано в иерархическое дерево ограничивающих объемов (BVH).

Вычислительный шейдер строит пирамиду глубин предыдущего кадра (Hi-Z buffer),

выполняет консервативную проверку пересечений ограничивающих параллелепипедов объектов с буфером глубины и отсекает невидимые меши на субпиксельном уровне до отправки команд растеризации, разгружая шину CPU-GPU.

В проектах на Unreal Engine 5.1 (*S.T.A.L.K.E.R. 2: Heart of Chernobyl*) стриминг столкнулся с аппаратным барьером декомпрессии. Даже накопители стандарта PCIe 4.0 NVMe упираются в узкое горлышко программного декодирования сжатых виртуальных текстур и микрополигональных кластеров Nanite силами центрального процессора, что приводит к рывкам времени кадра (traversal stuttering) при пересечении границ секторов карты.

Проект / Движок	Архитектура стриминга	Бюджет оперативной памяти под мир	Механизм сокрытия задержек I/O	Архитектурные ограничения и компромиссы
GTA III (RenderWare)	Скользящее окно, сектора по 2 КБ, зеркалирование DVD	~16–20 МБ из 32 МБ RAM PS2	Заслонки обзора, повороты улиц под 90°, туман	Ограничение скорости авто, запрет полетов, клоны трафика
Crash Bandicoot (NPT)	Кастомная файловая система, потоковые чанки по 64 КБ	До 2 МБ системного ОЗУ PS1	Жесткая рельсовая камера, поворот коридоров	Полная линейность, невозможность свободного обзора
GTA: San Andreas (RenderWare)	Зональный топологический стриминг	13.5 МБ (stream.ini)	Буферные пустынные и лесные зоны между городами	Агрессивный pop-in геометрии на самолетах, лимит памяти
Mass Effect 3 (UE3)	Блочная загрузка подуровней в память	~300 МБ из 512 МБ (PS3/X360)	Лифты, шлюзы декомпрессии, сканеры безопасности	Разрыв мира на изолированные арены, искусственные паузы
Batman: Arkham Knight (UE3 64-bit)	Потоковый асинхронный стриминг текстур и мешей	До 6–8 ГБ VRAM / RAM на ПК	Высотные полеты, пикирование, визуальный туман	Катастрофический статтеринг при быстрой езде на релизе
NFS: Rivals (Frostbite 3)	Выделенный сплайновый кэш коллизий трассы	До 2 ГБ ОЗУ под потоковые буферы	Скорость отрисовки LOD-импостеров вдалеке	Блокировка физического шага на 30 FPS, размытие окружения
Horizon Zero Dawn (Decima)	GPU-driven BVH culling, тороидальные клипмэпы	~4.5 ГБ GDDR5 памяти PS4	Иерархические HLOD-импостеры, динамический туман	Зависимость от очередей GPU compute, мерцание импостеров
S.T.A.L.K.E.R. 2 (UE5)	Nanite Virtualized Geometry + Virtual Shadow Maps	До 16 ГБ RAM и 10–12 ГБ VRAM	Пересеченный лесной ландшафт,	Нагрузка на ядра CPU при декомпрессии

Проект / Движок	Архитектура стриминга	Бюджет оперативной памяти под мир	Механизм сокрытия задержек I/O	Архитектурные ограничения и компромиссы
			погодные эффекты	ассетов с NVMe SSD

3. Процедурная генерация, воксельные миры и бесконечные пространства

Создание процедурных пространств требует принципиального изменения вычислительной модели: мир перестает храниться на физическом носителе как готовый массив вершин и полигонов и генерируется на лету с помощью математических алгоритмов детерминированного шума.

Историческим прорывом в процедурном дизайне открытых сред стал *Far Cry* (2004, CryEngine 1). Движок использовал концепцию полигонального ландшафта на основе карт высот (Heightmap Terrain) в сочетании со сжатыми воксельными нарезками для пещер и вертикальных скал. Разработчики реализовали динамический расчет LOD-сетки земли: ландшафт разбивался на квадратные блоки, и для каждого блока алгоритм непрерывно пересчитывал плотность треугольников в зависимости от расстояния до камеры. В *Far Cry 2* (2008) студия Ubisoft развила этот подход в движке Dunia Engine: в мир была интегрирована процедурная система распространения огня. Растительность и постройки нарезались на двумерную сетку горючести, где каждый узел отслеживал температуру, влажность и направление процедурного ветра, рассчитываемого по упрощенной модели Навье-Стокса, что позволяло пламени физически распространяться по саванне в реальном времени.

В воксельном сегменте мировой вехой стал *Minecraft* (2009/2011, Mojang), расколовшийся на две фундаментально различные архитектуры:

1. **Java Edition:** Мир генерируется из псевдослучайного 64-битного сида на основе трехмерного когерентного шума Перлина и разбивается на чанки размером 16×16 блоков по горизонтали и до 384 блоков по высоте. Каждый блок представляет собой байтовый идентификатор и метаданные. Главная архитектурная проблема Java Edition кроется в платформе JVM (Java Virtual Machine) и сборщике мусора (Garbage Collector). Выделение миллионов короткоживущих объектов при генерации новых чанков приводит к фрагментации кучи (heap) и неизбежным паузам GC (Stop-the-World), порождающим микрозаикания. Логика мира замкнута на один мастер-поток CPU, выполняющий тики симуляции с фиксированной частотой 20 TPS (Ticks Per Second, 50 мс на тик). При накоплении сущностей (мобы, редстоун-схемы) время тика превышает 50 мс, вызывая десинхронизацию физического времени с реальным. Отрисовка геометрии долгое время строилась на примитивных вызовах OpenGL с неэффективным перестроением вершинных буферов чанков.
2. **Bedrock Edition:** Полная переработка кодовой базы на C++ с рендер-конвейером RenderDragon (DirectX 12, Vulkan). Генерация и мешинг (триангуляция) воксельных чанков полностью вынесены в асинхронный пул рабочих потоков (Worker Threads). Для каждого чанка алгоритм выполняет жадную триангуляцию (Greedy Meshing), объединяя смежные грани одинаковых блоков в один четырехугольник, что снижает число полигонов на чанк на 60–80%. Память управляется вручную без сборщика мусора, что обеспечивает стабильные 60 кадров/с на смартфонах и дальность

прорисовки свыше 64–96 чанков против предельных 16–32 чанков в Java Edition. Масштабирование процедурной генерации до масштабов вселенной было реализовано в *No Man's Sky* (2016, Hello Games). Кастомный движок строит 18 квинтиллионов планет на базе единого 64-битного математического сида.

Архитектура опирается на Суперформулу Йохана Гиелиса — обобщение эллипса в полярных координатах, позволяющее с помощью десятка параметров генерировать бесконечное число сложных органических кривых и пространственных форм:

$$r(\varphi) = \frac{1}{\sqrt[n_1]{\left| \frac{\cos\left(\frac{m\varphi}{4}\right)}{a} \right|^{n_2} + \left| \frac{\sin\left(\frac{m\varphi}{4}\right)}{b} \right|^{n_3}}}$$

Ландшафт планет не является картой высот, а представляет собой непрерывное воксельное скалярное поле плотности, что дает возможность процедурно формировать нависающие скалы, пещеры и тоннели.

Для превращения воксельного поля в полигональную сетку движок использует алгоритм двойственного контурирования (Dual Contouring), исполняемый на вычислительных шейдерах GPU. Dual Contouring, в отличие от классического алгоритма Marching Cubes, анализирует градиенты плотности (эрмитовы данные) и находит точки пересечения поверхности внутри ячейки методом наименьших квадратов, что сохраняет острые геометрические грани скал при минимальном числе треугольников.

Стриминг планеты организован через квадродерево (Quadtree), вершины которого динамически расщепляются при приближении корабля к поверхности. Платой за математическую бесконечность стал дефицит уникальных ассетов, высокая нагрузка на видеочип при деформации ландшафта манипулятором терраформинга и необходимость агрессивного темпорального сглаживания для маскировки геометрического шума процедурных мешей.

Проект / Движок	Математическая основа генерации	Архитектура представления мира	Вычислительный баланс	Критические ограничения
Far Cry 2 (Dunia)	Процедурные карты высот + 2D-сетка горючести	Сплайновые дороги, полигональный террейн, воксельные скалы	CPU: расчет динамики огня и ветра; GPU: альфа-растительность	Отсутствие многослойных пещер, запеченная макротопология
Minecraft Java (JVM)	3D-шум Перлина / Симплекс-шум	Воксельная сетка чанков 16×16×384, поблочное хранение	Тяжелый CPU-bound на мастер-потоке, частые паузы GC кучи	Лимит 20 TPS, падение производительности при редстоун-сетях
Minecraft Bedrock (C++)	Симплекс-шум + многопоточный C++ мешинг	Воксели с жадной триангуляцией (Greedy Meshing)	Сбалансированный многопоточный CPU, разгрузка GPU	Упрощенный ИИ мобов, визуальные артефакты чанковых стыков
No Man's Sky (Собственный)	Суперформула Гиелиса + воксельный шум	Изоповерхности Dual Contouring на GPU, Quadtree-сферы	Экстремальная нагрузка на GPU Compute при входе в	Повторяемость паттернов флоры, шум на стыках LOD-сеток

Проект / Движок	Математическая основа генерации	Архитектура представления мира	Вычислительный баланс	Критические ограничения
			атмосферу	

4. Системная симуляция, физика, реактивные среды и искусственный интеллект

Интеграция честной физической симуляции и комплексного ИИ переносит вычислительный центр тяжести с видеокарты на ядра центрального процессора, требуя строгой изоляции тиков симуляции от графического рендеринга.

В *Half-Life 2* (2004, движок Source) Valve совершила революцию, внедрив лицензированный физический движок Havok 2 непосредственно в структуру геймплея. Главной инновацией стала сквозная табличная система субстанций (Substance System). В коде движка каждый интерактивный объект определялся физическим материалом: плотностью, массой, эластичностью, трением, плавучестью и хрупкостью. Это позволило создать гравитационную пушку, манипулирующую кинетическими импульсами тел. Симуляция опиралась на фиксированный шаг времени (fixed timestep, 66 Гц / 15 мс) с линейной интерполяцией положений объектов для графического кадра.

Чтобы процессор Pentium 4 не входил в ступор от тысяч одновременных расчетов коллизий, инженеры применили алгоритм Physics Sleeping: объект, чья линейная и угловая кинетическая энергия падала ниже порогового значения в течение нескольких кадров, выключался из контактного графа решателя (solver) и превращался в статическое тело до получения внешнего импульса.

В *Portal* (2007) и *Portal 2* (2011) Valve усложнила физическую модель Source задачей разрыва пространственной непрерывности. Портал представляет собой бесконечно тонкую плоскость с динамической заменой матрицы пространственной трансформации: $M_{\text{out}} = M_{\text{portal_exit}} \times M_{\text{portal_enter}}^{-1} \times M_{\text{in}}$. При частичном пересечении портала движок клонирует физическое тело объекта по другую сторону плоскости с сохранением вектора количества движения, пересчитывая контакты коллизий в двух разрозненных системах координат одновременно.

В *Portal 2* к этому добавилась симуляция динамических неньютоновских гелей (отталкивающего, ускоряющего и портового). Вместо тяжелого гидродинамического расчета частиц в реальном времени Valve применила запекание физических свойств в динамические текстурные развертки геометрии уровня: капли геля при падении растекались по сплайнам, на лету модифицируя коэффициенты трения и упругости поверхностных полигонов в таблицах материалов движка Source.

В серии *The Elder Scrolls* (*The Elder Scrolls IV: Oblivion*, 2006; *The Elder Scrolls V: Skyrim*, 2011, движки Gamebryo и Creation Engine) Bethesda сделала ставку на абсолютную персистентность мира. Каждый предмет в игре (от яблока до меча) является независимой сущностью с сохранением координат, массы и физических параметров Havok в глобальном файле сохранений.

Симуляция NPC опиралась на поведенческую систему Radiant AI: персонажи не привязаны к статичным точкам, а имеют суточные циклы потребностей (сон, голод, работа, общение), динамически выбирая цели на глобальном графе навигации (NavMesh). Архитектурной уязвимостью движка стала жесткая синхронизация физического таймера Havok с частотой кадровой развертки, зафиксированной на 60 Гц. При попытке

разблокировать частоту кадров на ПК до 120–144 FPS физический интегратор Верле получал неверную дельту времени (Δt), что вызывало хаотический взрыв коллизий: посуда и мебель разлетались по комнатам со сверхзвуковой скоростью, а персонажи погибали от микроударов об окружающие предметы.

В *The Legend of Zelda: Breath of the Wild* (2017) Nintendo надстроила над ядром Havok независимый «химический движок» (Chemistry Engine). Физика симулирует пространственные перемещения твердых тел, а химия управляет состояниями элементов и материалов. Система описывается конечным автоматом с жесткими правилами:

- Элементы (Огонь, Вода, Лед, Электричество, Ветер) взаимодействуют с материалами (Дерево, Металл, Камень, Ткань).
- Элементы реагируют между собой (вода тушит огонь, генерируя пар; лед тает от тепла, превращаясь в воду).
- Материалы не взаимодействуют напрямую с материалами (камень не плавит дерево).

Эта комбинаторная система породила спонтанный эмерджентный геймплей: поджигание травы создает восходящий поток нагретого воздуха, раскрывающий планер героя, а металлический меч, брошенный во врага во время грозы, притягивает удар молнии. Однако вычислительная слабость процессора Espresso консоли Wii U (три ядра PowerPC 750) приводила к исчерпанию 33.3 мс бюджета кадра при цепных реакциях горения, что вызывало ступенчатое падение частоты кадров с 30 до фиксированных 20 кадров/с из-за жесткой двойной буферизации V-Sync.

В глобальной стратегии *Hearts of Iron IV* (2016, Clausewitz Engine от Paradox Interactive) симуляция носит абстрактно-математический характер, создавая запредельную нагрузку на процессор в поздней фазе партии (проблема late-game lag). Движок каждый игровой час производит расчет снабжения, перемещения и боя сотен дивизий по десяткам тысяч провинций земного шара.

Поскольку дипломатические матрицы ИИ и алгоритмы поиска пути снабжения дивизий по графам железных дорог долгое время исполнялись в одном основном потоке логики, к 1943–1945 виртуальным годам время обработки суток возрастает с долей секунды до нескольких реальных секунд. Масштабирование на многоядерные процессоры блокировалось архитектурным требованием абсолютного сетевого детерминизма: малейшее расхождение порядка выполнения операций с плавающей запятой между клиентами приводит к мгновенной рассинхронизации партии (Out of Sync).

В шутере *Sniper Elite 5* (2022, Asura Engine от Rebellion) фокус симуляции смещен на анатомическую систему X-Ray Kill Cam. Движок рассчитывает баллистику пули с учетом гравитационного спада, деривации вращения нареза ствола и силы бокового ветра. При попадании игра переключается в замедленный режим: включается многослойная физическая модель человеческого тела, содержащая процедурные скелетные кости с переменной плотностью, внутренние органы и кровеносную систему. Попадание пули порождает динамическую временную кавитационную полость на основе уравнений деформации упругих тел, разрушая кости на процедурные осколки и разрывая мягкие ткани, что требует мгновенного перераспределения вычислительных мощностей с рендера окружения на физическую симуляцию локального объема.

Катастрофическим примером кризиса симуляции толпы стал релиз *Dragon's Dogma 2* (2024, RE Engine от Capcom). В открытых пространствах движок демонстрировал высокую производительность, но в столице Вернворт кадровая частота падала до 25–35 кадров/с даже на процессорах Intel Core i9-14900K и AMD Ryzen 7 7800X3D вне зависимости от видеокарты и разрешения.

Инженеры Digital Foundry выяснили, что RE Engine в реальном времени симулирует полный цикл логики, физику тканей и волос, поиск пути NavMesh и зрение для каждого горожанина и пешки (Pawn) в радиусе города одновременно, без разделения на потоки и без динамического масштабирования нагрузки. Вычислительный цикл замыкался на одном мастер-потоке CPU, а видеокарта уровня GeForce RTX 4090 простаивала с загрузкой 40–50%. Ситуацию удалось частично стабилизировать лишь через полгода введением ручной настройки плотности населения (Character Population Density) и переносом части задач ИИ в асинхронные пулы.

Проект	Движок / API	Алгоритмический базис	Характер вычислительной нагрузки	Архитектурные методы стабилизации
Half-Life 2 (2004)	Source / Havok 2	Таблицы субстанций, фиксированный шаг 66 Гц	Умеренная однопоточная нагрузка CPU	Спячка тел Physics Sleeping, грубые коллизионные боксы
Portal 2 (2011)	Source / Havok	Рекурсивные порталные матрицы, запекание гелей	Всплески draw calls при просмотре порталов	Ограничение глубины рекурсии трафаретного буфера (N=2)
Skyrim (2011)	Creation / Havok	Radiant AI, глобальный граф NavMesh, персистентность	CPU: физические коллизии сотен интерактивных предметов	Жесткий локап на 60 FPS во избежание разрушения интегратора
Zelda: BotW (2017)	Кастомный + Havok	Матрица стихийного химического движка	Резкие скачки времени кадра при цепных пожарах	Пропуск тактов распространения пламени, ступенчатый сброс до 20 FPS
Hearts of Iron IV (2016)	Clausewitz	Дискретный почасовой пересчет графов снабжения	Критический CPU Single-Thread упор в late-game	Упрощение логистических узлов, агрегирование дивизий ИИ
Sniper Elite 5 (2022)	Asura Engine	Внешняя баллистика + деформация кавитационных полостей	Пиковая нагрузка на CPU/GPU во время X-Ray замедления	Заморозка симуляции окружающего мира на время Kill Cam
Dragon's Dogma 2 (2024)	RE Engine	Индивидуальный ИИ сотен горожан и пешек в реальном времени	Блокировка мастер-потока CPU в Вернворте (30 FPS ceiling)	Введение ползунка плотности толпы, частичный параллелизм

5. Сетевые архитектуры, соревновательный

детерминизм и серверная инфраструктура

Сетевой код соревновательных игр решает фундаментальную проблему: скорость распространения сигнала в оптоволокне конечна, из-за чего клиенты наблюдают игровое состояние с задержкой в 20–150 мс.

В соревновательных шутерах классическим стандартом стала клиент-серверная модель с компенсацией запаздывания (Lag Compensation). В *Counter-Strike: Global Offensive* (2012, Source) сервер квантовал время фиксированными тиками (64 Гц в матчмейкинге, 128 Гц на турнирах). При выстреле сервер считывал сетевую задержку стрелка и интерполяционный буфер (cl_interp), физически перематывал хитбоксы всех соперников в координатном пространстве назад во времени на величину сетевого пинга стрелка, производил расчет пересечения траектории пули лучом и возвращал мир в текущий момент времени. Это обеспечивало безупречную точность стрельбы для атакующего, но порождало неустрашимый артефакт «смерти за углом» (Peeker's Advantage): игрок, успевший забежать за бетонное укрытие на своем экране, погибал, так как на экране оппонента с высоким пингом его проекция все еще находилась на открытой линии огня. В 2023 году Valve в *Counter-Strike 2* (Source 2) внедрила архитектуру Sub-Tick. Инженеры отказались от дискретного квантования действий strictly по границам тиков. Теперь клиент при нажатии клавиши выстрела или прыжка отправляет пакет с точным микротаймстемпом момента ввода внутри текущего кадра. Сервер вычисляет баллистику ровно в тот субмомент времени, когда был совершен клик.

Однако базовая рассылка снапшотов мира клиентам осталась на частоте 64 Гц. На старте это породило когнитивный диссонанс: выстрел регистрировался сервером между тиками, а анимация выстрела, вспышки и падения рэгдолла воспроизводилась клиентом только со следующим полученным снапшотом (через 10–15 мс), что воспринималось игроками как «ватный» и нечеткий отклик управления.

В командном шутере *Overwatch* (2016, Blizzard) сетевой стек строился на детерминированной симуляции на базе архитектуры ECS (Entity Component System) с высокой частотой обновления (63 Гц на клиенте, до 120 Гц в соревновательных режимах). Сетевой код закрепил принцип Favor the Shooter: сервер валидирует урон со стороны стреляющего, если цель была в прицеле на его локальном экране.

Исключение составили способности мгновенной неуязвимости (скачок Трейсер, ледяная глыба Мэй, рывок Гендзи): для них внедрен наивысший сетевой приоритет с локальной отменой полученного урона при совпадении таймингов. Переход от формата 6v6 к 5v5 в *Overwatch 2* диктовался в том числе сетевой экономикой: одновременная синхронизация двенадцати персонажей с постоянным спамом щитов, лучевых атак и аур создавала избыточный сетевой трафик и перегружала серверные экземпляры расчетами пересечения объемных коллайдеров.

В жанре файтингов задержка даже в один кадр (16.6 мс) разрушает тайминги комбо-ударов. Провалом завершился релиз *Street Fighter V* (2016), где сетевой rollback-код Capcom страдал от рассинхронизации часов (Clock Drift): при просадке кадров у одного игрока игра накапливала односторонний лаг, порождая дикие телепортации персонажей у второго участника матча.

Индустриальным эталоном стал *Guilty Gear Strive* (2021, Arc System Works). Сетевой код создавался на основе технологии детерминированного отката (Deterministic Rollback).

Игровое состояние (хитбоксы, шкалы здоровья, координаты) было полностью изолировано от визуального конвейера Unreal Engine 4 (скелетные меши, шейдеры,

частицы).

При получении запоздавшего сетевого пакета от оппонента движок за доли миллисекунды откатывает логическое состояние мира на K кадров назад, накатывает запоздавший ввод, мгновенно пересчитывает физику за пропущенный отрезок и продолжает игру без паузы. Чтобы слабый процессор PlayStation 4 справлялся с многократным пересчетом положений костей скелета за один кадр, математический блок трансформации матриц был переписан под векторные SIMD-инструкции, обеспечив идеальный отклик при пинге до 150–200 мс. Специфический сетевой вызов представляет физический мультиплеер в *Fall Guys* (2020, Unity). Сервер должен синхронизировать перемещения до 60 игроков, каждый из которых представляет собой нестабильное рэгдолл-тело на базе PhysX, непрерывно сталкивающееся с вращающимися препятствиями и десятками других игроков. Попытка полной серверной авторитарной симуляции 60 рэгдоллов приводила к исчерпанию тикрейта. Mediatonic развернула гибридную модель: физика положения собственного персонажа рассчитывается локально на клиенте и отправляется на сервер, а сервер выступает валидатором пересечений коллизий и транслирует интерполированные векторы другим участникам. При скоплении толпы на узких мостах система принудительно смягчает взаимные коллизии между игроками, переводя персонажей в полупроницаемое состояние, что исключает разлет физических тел в сингулярность.

В многопользовательских мирах барьером выступает квадратичный рост трафика:

$$P = O(N^2)$$

где N — число игроков в локации. В *World of Warcraft* (Blizzard) скопление сотен игроков в одной зоне (как при открытии Врат Ан'Киража в *WoW Classic*) вызывало падение серверного тикрейта до долей герца. Сервер перегружался перекрестным оповещением клиентов о смене координат и направлений взгляда.

Blizzard решила проблему комплексно:

- Протоколы деградации: при критической плотности сервер прекращает рассылать векторы поворота персонажей (facing-updates) и второстепенные ауры, снижая частоту тиков зоны.
- Пространственный шардинг (Sharding) и лееринг (Layering): единый мир прозрачно дробится на виртуальные изолированные слои с балансировкой игроков между физическими серверами кластера, исключая перегруз одной локации ценой внезапного исчезновения союзников прямо во время боя.

Проект	Сетевая топология	Модель синхронизации	Поведение при лагах и потерях	Системные компромиссы и артефакты
CS:GO (2012)	Клиент-сервер (64/128 Гц)	Lag Compensation, перемотка хитбоксов назад	Клиентская экстраполяция, сглаживание дельт	«Смерть за углом», высокая нагрузка на сервер при 128 тиках
CS2 (2023)	Клиент-сервер (Sub-Tick + 64 Гц снапшоты)	Непрерывная регистрация таймстемпов ввода	Субмоментный расчет точек баллистики	Визуальное запаздывание рэгдолла и анимаций до границы тика
Overwatch 2	Клиент-сервер на	Favor the Shooter	Агрессивное	Смерть в рывке

Проект	Сетевая топология	Модель синхронизации	Поведение при лагах и потерях	Системные компромиссы и артефакты
(2022)	базе ECS (63–120 Гц)	с приоритетом неуязвимостей	сглаживание траекторий, локальный хит	при высоком пинге, ограничение матчей 5v5
Guilty Gear Strive (2021)	Peer-to-Peer	Детерминированный Rollback, изоляция стейта	Мгновенный локальный ввод, перемотка на К кадров	Резкие микротелепортации спрайтов при скачках пинга >200 мс
Fall Guys (2020)	Авторитарный клиент-сервер	Физическая интерполяция рэгдоллов PhysX	Локальное предсказание движения, мягкие коллизии	Рассинхронизация захватов хвостов, расталкивание в толпе
World of Warcraft (Кластер)	Распределенный серверный кластер	Динамический шардинг и лееринг зон	Деградация тикрейта, отсечение второстепенных пакетов	Исчезновение мобов и игроков при пересечении границ шардов

6. Системная инженерия ПК-портов, низкоуровневые API и аппаратные барьеры

Низкоуровневые API (DirectX 12, Vulkan) сняли накладные расходы драйвера, но возложили управление видеопамью, барьерами синхронизации и компиляцией конвейера на плечи игровых программистов.

Главным бичом современных игр стал компиляционный статтеринг (Shader Compilation Stuttering). В DirectX 11 видеодрайвер компилировал шейдеры динамически и сглаживал всплески фоновым кэшированием. В DirectX 12 появилась монолитная концепция Pipeline State Object (PSO): шейдеры, форматы буферов, режимы смешивания, тест глубины и трафарета должны быть скомпилированы в единый неизменяемый бинарный блок строго до вызова команды отрисовки. Число комбинаций PSO в крупных проектах достигает сотен тысяч. Если движок пытается скомпилировать отсутствующий PSO в реальном времени (Just-In-Time компиляция), главный графический поток CPU блокируется на 30–250 мс, вызывая жесткий стоп-кадр.

Катастрофическим примером стал ПК-релиз *The Last of Us Part I* (2023, Iron Galaxy / Naughty Dog). Движок, заточенный под фиксированное железо PlayStation 5, попытался скомпилировать гигантскую базу PSO при первом запуске на ПК. Это приводило к 100% загрузке всех ядер CPU в течение нескольких часов, перегреву кристаллов, утечкам системной памяти и вылетам по Out-of-Memory.

Схожий кризис пережил *Elden Ring* (2022, FromSoftware): отсутствие предварительного прогрева конвейера (PSO Warmup) вызывало заикания при каждом новом заклинании или эффекте босса. Valve решила эту проблему для портативной консоли Steam Deck на уровне операционной системы, распространяя предварительно скомпилированный

монолитный кэш шейдеров (Fossilize) через сервера Steam прямо на устройства пользователей.

В *Star Wars Jedi: Survivor* (2023, Unreal Engine 4) на релизе сложился комплексный технический тупик:

- Отсутствие полноценного предварительного прогрева PSO вызывало статтеры при исследовании локаций.
- Неэффективное построение и обновление иерархии BVH для трассировки лучей в хаб-локациях (планета Кобо) парализовало очереди команд DX12.
- Движок перегружал главный рендер-поток тяжелыми графами сценариев Blueprint, в результате чего даже связка из Core i9-13900K и RTX 4090 не могла преодолеть планку 40–50 кадров/с в городе из-за глухого упора в одно ядро процессора.

В *The Witcher 3: Wild Hunt* (Next-Gen обновление 2022 года, REDengine 3) разработчики внедрили трассировку лучей и DirectX 12 поверх старого движка 2015 года. Вместо нативной интеграции низкоуровневого API был применен слой трансляции (DX11-to-DX12 wrapper). В Новиграде и Боклере частота кадров рушилась на 40–50% даже без включения трассировки лучей исключительно из-за того, что транслятор генерировал неэффективные барьеры синхронизации, блокируя процессорные ядра в бесконечных ожиданиях GPU. Обойти этот барьер на ПК позволила лишь генерация кадров DLSS 3, интерполирующая кадры поверх перегруженного процессорного потока.

Корни однопоточного процессорного кризиса уходят в конец 2000-х годов, когда тактовая частота процессоров уперлась в тепловой барьер ~3.8–4 ГГц, вынудив чипмейкеров наращивать число параллельных ядер. Игры с однопоточным главным циклом оказались в исторической ловушке. ПК-релиз *Grand Theft Auto IV* (2008, RAGE) сочетал процедурную симуляцию эйфорических рэздоллов (Euphoria) и отложенный рендеринг, но цикл симуляции мира и вызовы DirectX 9 замыкались на одном мастер-ядре CPU. Видеокарты простаивали с загрузкой 30–40%, пока ядро процессора троттлоло от перегрузки.

В *Forza Horizon 3* (2016, ForzaTech) студия Playground Games совершила ту же ошибку: рендеринг намертво блокировал ядро Core 0. Лишь в *Forza Horizon 4* (2018) движок подвергся полной перестройке с внедрением параллельной архитектуры задач (Task-Based Job System), распределяющей симуляцию и отрисовку равномерно по всем ядрам.

В сегменте 2D-инди высочайший уровень системной дисциплины продемонстрировали *Hollow Knight* и *Hades*:

- В *Hollow Knight* (2017, Unity) студия Team Cherry столкнулась с ограничениями сборщика мусора Mono/C# на портативной консоли Nintendo Switch. Чтобы исключить просадки времени кадра от срабатывания Garbage Collector, разработчики полностью исключили динамическую аллокацию памяти в боевых циклах (GC Alloc = 0): был внедрен пулинг объектов для всех частиц, снарядов и мобов. Физика перемещения Рыцаря была отвязана от стандартного движка PhysX2D и построена на прямых лучах (Raycasts) с дискретным разрешением коллизий, что обеспечило бескомпромиссный отклик управления без накладных расходов.
- В *Hades* (2020) Supergiant Games применила собственный движок на базе C++ и кастомного слоя скриптования. Для обеспечения динамичного экшена при 60 кадрах/с на слабых системах движок оперирует гигантскими атласами сжатых 2D-спрайтов высокого разрешения с аппаратной декомпрессией, гарантируя нулевые паузы памяти во время массовых атак десятков противников.

В современных AAA-проектах спасением от системных ограничений стали темпоральные

алгоритмы и реконструкция данных. В ПК-версии *Ghost of Tsushima* (2024, Nixxes) разработчики впервые в индустрии программно развязали апскейлер и интерполятор, позволив активировать генератор кадров AMD FSR 3 поверх базового масштабирования NVIDIA DLSS Super Resolution. Это позволило обойти артефакты субпиксельного шиммеринга тонкой растительности и веток бамбука, обеспечив качество сглаживания и стабильность картинки выше нативного рендеринга в 4K при двукратном росте плавности вывода.

Проект	Движок / API	Корневой системный дефект	Поведение на релизе	Методы устранения и обхода
GTA IV (2008)	RAGE / DX9	Монолитный однопоточный цикл мира	Статтеры, неполная загрузка GPU	Архитектурно не исправлено; обход через грубую мощь IPC CPU
Batman: Arkham City (2011)	UE3 / DX11	Нестабильный ранний конвейер DX11, утечки PhysX	Вылеты, заикания тесселяции на 32-битных ОС	Патчи очередей команд, перенос части PhysX расчетов на CPU
Forza Horizon 3 (2016)	ForzaTech / DX12	Завязка конвейера на Core 0 CPU	Упор в 1-е ядро, 35–45 FPS на топовых видеокартах	Переход к параллельной Job System в <i>Forza Horizon 4</i>
Elden Ring (2022)	Собственный / DX12	JIT-компиляция состояний PSO в реальном времени	Стоп-кадры до 250 мс при появлении новых эффектов	Прекомпиляция монолитного кэша силами Steam Deck / драйверов
The Last of Us Part I (2023)	Собственный / DX12	Неуправляемый размер кэша PSO, утечки памяти	Часы компиляции шейдеров, вылеты по памяти OOM	Глубокие патчи стриминга и сжатие базы конвейеров
SW Jedi: Survivor (2023)	UE4 / DX12	CPU-bound в хабах, нагромождение RT BVH, JIT PSO	35–40 FPS на флагманских CPU, постоянные рывки	Отключение трассировки, патчи распределения потоков
Hollow Knight (2017)	Unity / Mono	Риск пауз сборщика мусора (GC) на Nintendo Switch	Потенциальные микрофризы при спавне ассетов	Архитектура Zero GC Alloc в бою, raycast-физика
Ghost of Tsushima (2024)	Собственный / DX12	Шиммеринг субпиксельной травы при нативном TAA	Дрожание листвы, мыло при быстром движении	Независимая связка DLSS Super Resolution + FSR 3 Frame Gen

7. Комплексный профиль системных ресурсов для СППР

Для интеграции исследованных проектов и методов в инженерную систему поддержки принятия решений (СППР) ниже сведена калибровочная матрица влияния технологий на ресурсы целевых систем по шкале от -3 до +3, где:

- отрицательные значения (-3 \dots -1) означают экономию или разгрузку ресурса;
- положительные значения (+1 \dots +3) означают прирост потребления или нагрузку на подсистему;
- 0 означает нейтральное влияние.

Матрица охватывает ключевые подсистемы: **CPU** (вычислительные ядра процессора), **GPU** (растровые и шейдерные блоки), **RAM** (системная память), **VRAM** (видеопамять), **Disk** (нагрузка на дисковый I/O и шину PCIe) и **Net** (объем сетевого трафика).

Метод / Технологическое решение	CPU	GPU	RAM	VRAM	Disk	Net	Ключевые проекты-бенчмарки	Архитектурный trade-off (компромисс)
2D BSP-отсечение видимости	-2	-3	-3	-3	-2	0	<i>Doom</i> (1993)	Экстремальная скорость ценой запрета многоэтажности
ТМЕМ-микротекстурирование	-1	-2	-3	-3	-2	0	<i>Super Mario 64</i> (1996)	Работа в 4 КБ кэша ценой сильного размытия текстур
Блочный стриминг секторов (2 КБ)	-1	0	-3	-2	+3	0	<i>GTA III</i> , <i>Crash Bandicoot</i>	Экономия RAM ценой лимита скорости перемещения
Зональное топологическое буферизирование	0	0	-2	-2	+2	0	<i>GTA: San Andreas</i>	Бесшовный мир ценой введения пустых буферных зон

Метод / Технологическое решение	CPU	GPU	RAM	VRAM	Disk	Net	Ключевые проекты-бенчмарки	Архитектурный trade-off (компромисс)
Разгрузка декомпрессии на SPU/DMA	-2	0	-1	-1	+2	0	<i>Red Dead Redemption</i> (PS3)	Разгрузка CPU ценой сложнейшего микрокода под Cell
Hierarchical LOD + Terrain Clipmaps	+1	+1	+1	+1	+2	0	<i>GTA V</i> , <i>Red Dead Redemption 2</i>	Бескрайний горизонт ценой фоновой декомпрессии
Декомпрессионные шлюзы (лифты)	-2	-2	-2	-2	+1	0	<i>Mass Effect 3</i>	Стабильность в 512 МБ ОЗУ ценой задержки игрока
Физический сплайновый кэш трассы	+1	0	+1	0	+1	0	<i>Need for Speed</i> (Rivals/MW)	Честная физика колес ценой pop-in визуальных ассетов
GPU-driven BVH culling (Decima)	-2	+1	0	0	+1	0	<i>Horizon Zero Dawn</i>	Разгрузка CPU ценой нагрузки на GPU compute
Nanite + Virtual Shadow Maps	+2	+3	+2	+2	+3	0	<i>S.T.A.L.K.E.R. 2</i> , <i>Black Myth</i>	Кинематографичный меш ценой traversal-статтеров I/O

Метод / Технологическое решение	CPU	GPU	RAM	VRAM	Disk	Net	Ключевые проекты-бенчмарки	Архитектурный trade-off (компромисс)
Deferred Shading + SH-зонды	-1	+1	0	+1	0	0	<i>MGS V: The Phantom Pain</i>	Сложность $O(N+M)$ ценой проблем с прозрачностью
Clustered Forward Rendering	+1	+1	0	-3	0	0	<i>DOOM Eternal</i>	Экономия сотен МБ VRAM ценой сложной синхронизации
Мобильный NPR Cel-Shading (TBDR)	-1	-1	-2	-2	-1	0	<i>Genshin Impact</i>	Отсутствие перегрева мобильных GPU ценой упрощения света
Ретро-папплайн (Vertex Snapping)	-2	-3	-3	-3	-2	0	<i>Ultrakill</i>	Минимальный инпут-лаг ценой дрожания вершин и UV
DDGI Infinite Bounces (Ray Tracing)	+1	+3	0	+2	0	0	<i>Metro Exodus EE</i>	Отказ от запекания ценой инерционного темпорального лага
ReSTIR Path Tracing + DLSS RR	+1	+3	+1	+2	0	0	<i>Cyberpunk 2077, Wukong</i>	Честные фотоны ценой обязательств

Метод / Технологическое решение	CPU	GPU	RAM	VRAM	Disk	Net	Ключевые проекты-бенчмарки	Архитектурный trade-off (компромисс)
								ьного апскейлинга
Mesh Shaders Geometry Pipeline	-1	+2	0	+1	0	0	<i>Alan Wake 2</i>	Субпиксельный каллинг ценой отказа от старых видеокарт
Процедурный шум + Dual Contouring	+2	+3	+1	+1	-3	0	<i>No Man's Sky</i>	Бесконечные миры ценой высокой нагрузки на GPU compute
Воксельная триангуляция (JVM)	+3	0	+3	0	+1	0	<i>Minecraft Java</i>	Свобода модификаций ценой пауз сборщика мусора GC
Многопоточный мешинг (C++/DX12)	+1	+1	-1	0	+1	0	<i>Minecraft Bedrock</i>	60 FPS на смартфонах ценой упрощения внутренней логики
Сплайн-запекание гелей в текстуры	+1	0	0	+1	0	0	<i>Portal 2</i>	Имитация неньютоновских жидкостей без флюид-с

Метод / Технологическое решение	CPU	GPU	RAM	VRAM	Disk	Net	Ключевые проекты-бенчмарки	Архитектурный trade-off (компромисс)
								иммуляции
Табличная система субстанций	+2	0	+1	0	0	0	<i>Half-Life 2</i>	Интерактивность ценой спячки Physics Sleeping
Персистентность объектов (Radiant)	+2	0	+2	0	+1	0	<i>Skyrim, Oblivion</i>	Живой мир ценой жесткой привязки физики к 60 FPS
Химический движок стихий (BotW)	+3	0	+1	0	0	0	<i>Zelda: Breath of the Wild</i>	Эмерджентность ценой просадок до 20 FPS при пожарах
Детерминированный расчет дивизий	+3	0	+2	0	0	0	<i>Hearts of Iron IV</i>	Масштаб ценой жесткого late-game lag в одном потоке CPU
Анатомический X-Ray кавитационный расчет	+2	+1	+1	0	0	0	<i>Sniper Elite 5</i>	Реализм ранений ценой паузы симуляции и фонового мира
Сквозной ИИ сотен горожан (Uncarpe	+3	0	+1	0	0	0	<i>Dragon's Dogma 2</i>	Сложное поведение NPC ценой упора в

Метод / Технологическое решение	CPU	GPU	RAM	VRAM	Disk	Net	Ключевые проекты-бенчмарки	Архитектурный trade-off (компромисс)
d)								мастер-поток CPU
Lag Compensation (Перемотка тиков)	+1	0	0	0	0	+1	CS:GO	Точность стрельбы ценой «смертей за углом»
Sub-Tick баллистическая дискретизация	+2	0	0	0	0	+2	Counter-Strike 2	Субмоментная регистрация ценой рассинхрона анимаций
Детерминированная ECS (Favor Shooter)	+1	0	0	0	0	+1	Overwatch 2	Честность дуэлей ценой сетевого сокращения команд до 5v5
SIMD Deterministic Rollback	+2	0	0	0	0	-2	Guilty Gear Strive	Нулевой сетевой лаг ценой быстрой ресимуляции костей
Клиент-серверные рэгдоллы с интерполяцией	+2	0	+1	0	0	+2	Fall Guys	60 физ-тел ценой полупроницаемых коллизий в толпе
Динамический шардинг и лееринг	+2	0	+2	0	0	-1	World of Warcraft	Преодоление барьера $O(N^2)$ ценой

Метод / Технологическое решение	CPU	GPU	RAM	VRAM	Disk	Net	Ключевые проекты-бенчмарки	Архитектурный trade-off (компромисс)
								разрыва групп игроков
Zero GC Alloc + Raycast 2D Physics	-2	-2	-2	-2	-1	0	<i>Hollow Knight</i>	Абсолютная стабильность времени кадра ценой ручного пулинга
Аппаратные атласы сжатых спрайтов	-1	+1	+1	+1	+1	0	<i>Hades</i>	Моментальный ввод ценой повышенного расхода видеопамати
Pre-warmed Monolithic с PSO Caching	+3	0	+2	+1	+1	0	<i>The Last of Us Part I, Elden Ring</i>	Полное отсутствие микрофризов ценой долгого первого старта
Task-Based Job System (Параллелизм)	-2	0	0	0	0	0	<i>Forza Horizon 4</i>	Идеальное масштабирование CPU ценой сложности архитектуры
Независимый	-1	+2	0	+1	0	0	<i>Ghost of Tsushima</i>	Плавность

Метод / Технологическое решение	CPU	GPU	RAM	VRAM	Disk	Net	Ключевые проекты-бенчмарки	Архитектурный trade-off (компромисс)
DLSS SR + FSR 3 Frame Gen								картинки ценой накладных расходов на инпут-лаг

8. Архитектурные паттерны и системное влияние на индустрию

Эволюция игровых движков подчиняется циклическому закону: как только программная модель исчерпывает возможности доступного аппаратного обеспечения, индустрия совершает парадигмальный сдвиг в алгоритмах.

1. **Эпоха дефицита памяти и медленных носителей (1990-е — середина 2000-х годов):** *Doom*, *Super Mario 64*, *Crash Bandicoot*, *GTA III* и *GTA: San Andreas* доказали, что предельные ограничения аппаратуры (лимит ОЗУ в единицы мегабайт, пропускная способность приводов CD/DVD в сотни килобайт) формируют базовый дизайн виртуальных пространств. Отсутствие памяти под Z-буфер породило BSP-деревья, лимит кэша TMEM сформировал облик низкополигонального 3D-платформинга, а физическая скорость перемещения лазерной головки DVD-привода определила планировку виртуальных улиц и скорость передвижения транспорта.
2. **Кризис однопоточного быстродействия и расцвет динамического света (конец 2000-х — 2010-е годы):** Остановка роста тактовой частоты процессоров на рубеже 3.8 ГГц и переход к многоядерным кристаллам застали индустрию врасплох. Проекты, полагавшиеся на грубую мощь одного мастер-потока (*GTA IV*, *Crysis*, *Forza Horizon 3*, *Hearts of Iron IV*), оказались в заложниках собственных архитектур. Одновременно с этим переход к отложенному освещению (G-буферы в *MGS V*) снял вычислительный барьер прямого рендеринга, но создал новый кризис перерасхода видеопамати при росте экранных разрешений до 4K. Ответом на этот вызов стал возврат к кластеризованному прямому рендерингу (*DOOM Eternal*) и повсеместное внедрение асинхронных многопоточных графов задач (Job Systems).
3. **Эпоха аппаратной трассировки, стохастических симуляций и машинного обучения (2020-е годы):** Современный этап знаменует окончательный отказ от запекания статических данных в пользу физически корректных методов Монте-Карло в реальном времени. Внедрение алгоритмов ReSTIR в *Cyberpunk 2077* и *Black Myth: Wukong*, сеточных конвейеров Mesh Shaders в *Alan Wake 2* и нейросетевой реконструкции лучей превратило графический процессор в суперкомпьютер пространственно-временных вероятностных вычислений. В то же время низкоуровневые API обнажили цену инженерной ошибки: отсутствие контроля над состояниями конвейера PSO (*The Last of Us Part I*, *Jedi: Survivor*) или отказ от

бюджетирования симуляции толп (*Dragon's Dogma 2*) способны парализовать даже передовые аппаратные конфигурации.

Главный инженерный вывод полувековой эволюции видеоигровых систем формулируется однозначно: стабильность времени кадра и глубина интерактивности определяются не грубой полигональной плотностью, а архитектурной дисциплиной — способностью движка изолировать подсистемы, вовремя отсекаать симуляцию невидимого пространства, управлять задержками шин ввода-вывода и закладывать контролируруемую деградацию точности при пиковых нагрузках.

Источники

1. AMD Radeon RX 7600 vs Nvidia GeForce RTX 3060 vs Intel Arc A750, <https://www.digitalfoundry.net/articles/digitalfoundry-2023-amd-radeon-rx-7600-vs-nvidia-rtx-3060-vs-arc-a750?page=7> 2. How to improve FPS in Dragon's Dogma 2 (PC) - CheckFPS, <https://checkfps.io/en/optimize-fps/dragons-dogma-2> 3. A new update for Dragon's Dogma 2 is available now on all ... - Reddit, https://www.reddit.com/r/DragonsDogma/comments/1dpfe3i/a_new_update_for_dragons_dogma_2_is_available_now/ 4. Dragon's Dogma 2 performance analysis: It'll take everything your, https://www.reddit.com/r/Games/comments/1bjg5hg/dragons_dogma_2_performance_analysis_itll_take/ 5. Dragon's Dogma 2 is a Mess: GPU & CPU Benchmarks, Bottlenecks, https://www.reddit.com/r/nvidia/comments/1bnatr4/dragons_dogma_2_is_a_mess_gpu_cpu_benchmarks/